Parallel Programming in Linux Using OpenMP

OpenMP is a versatile, open source paradigm for parallel programming. Learn how to use it to run processes simultaneously and tackle large data computations.

The large amount of data being generated today requires a lot of computation for analysis. As a result, our computing needs have increased exponentially. Parallel computing has been a topic of wide research for a few years now. It basically means breaking down a large problem into independent small problems and running these simultaneously using multiprocessors, and also communicating through shared memory. The solutions from each of these processors are then combined to give a final result.

OpenMP is one such paradigm used for parallel programming. It is an open source programming interface that supports different kinds of multiprocessing, shared memory, etc, and that too in different programming languages like C++, C, Fortran, and more! It is also compatible with multiple operating systems like Linux, Mac, AIX, Solaris, Windows, and others. It is very scalable and flexible.

Now that we understand what OpenMP is and what it does, let us look into how we can use it.

As a first step, open up a blank text editor and paste the following code:

```
#include<stdio.h>
#include<omp.h>
 int main()
{
#pragma omp parallel
printf("Hello\n"); }
```

Here, we first import the required headers, '*omp.h*' and '*stdio.h*'. In the main code, we write '*#pragma omp*' with the parallel keyword. This will run the code below it in parallel on different threads.

Now save this text editor with a file name '*.c*'.

The next step is to open up a terminal and type the commands as shown below:

```
gcc -o filenamec -fopenmp filename.c
```

We will compile the code that we have written.

Next, we specify to the OS to what extent we want to parallelise our code, i.e., how many threads we want. This is done using the following command:

```
export OMP_NUM_THREADS = 2
```

We can now run our compiled code using the following command, using *./filename:*

```
(base) jishnu@jishnu-Lenovo-G50-70:~/
Desktop$ gcc -o helloic -fopenmp
hello1.c
(base) jishnu@jishnu-Lenovo-G50-70:~/
Desktop$ ./hello1c
Hello
Hello
(base) jishnu@jishnu-Lenovo-G50-70:~/
Desktop$
```

As you can see, it shows that we printed 'Hello' twice, once from each thread! That's pretty cool, right?

So, now let us see which thread is doing what. Open up the text editor and paste the following code:

```
#include <omp.h>
#include<stdio.h>
int main (){
int nthreads, tid;

#pragma omp parallel private(tid){
tid = omp_get_thread_num();
printf("Hello World from thread =
%d\n", tid);
/* Only master thread does this */
if (tid == 0)
 { nthreads = omp_get_num_threads();
printf("Number of threads = %d\n",
```